

Name: Suraj Singh

BTech Cse, 3rd year

sec:"B"

Cu2402561579

Roll no. = 59

Experiment No. 5

Experiment Name

Interactive Dashboard Development using Tableau / Microsoft Power BI

Aim

To design and develop an interactive business dashboard using Tableau or Microsoft Power BI for analyzing real-world datasets and presenting key performance indicators (KPIs) to support data-driven decision-making.

Github Link: [https://github.com/Suraj2007-del/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement-](https://github.com/Suraj2007-del/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement-Technical/blob/main/Mastery%20in%20Data%20Analytics%20and%20Visualizations%20and%20Career%20Advancement%20(Technical)/Experiment5/Experiment%205.py)

[Technical/blob/main/Mastery%20in%20Data%20Analytics%20and%20Visualizations%20and%20Career%20Advancement%20\(Technical\)/Experiment5/Experiment%205.py](https://github.com/Suraj2007-del/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement-Technical/blob/main/Mastery%20in%20Data%20Analytics%20and%20Visualizations%20and%20Career%20Advancement%20(Technical)/Experiment5/Experiment%205.py)

```
!pip install plotly ipywidgets -q

import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from ipywidgets import interact, widgets
from IPython.display import display, HTML
```

```
from google.colab import files
import pandas as pd
import numpy as np

# Ye button dikhayega, apni CSV file yahan se upload karo
uploaded = files.upload()

# Uploaded file ka naam automatically pakad lega
filename = list(uploaded.keys())[0]

df = pd.read_csv(filename, parse_dates=['Order_Date'])
```

```
df['Month'] = df['Order_Date'].dt.to_period('M').astype(str)

print(df.shape)
df.head()
```

Choose Files sample_su..._dataset.csv

sample_superstore_dataset.csv(text/csv) - 42819 bytes, last modified: 9/7/2026 - 100% done

Saving sample_superstore_dataset.csv tosample_superstore_dataset.csv

(500, 11)

	Order_ID	Order_Date	Region	Category	Sub_Category	Sales	Quantity	Discount	Profit	Customer_Age	Month	
0	ORD-1000	2022-01-01 00:00:00.000000	North	Technology	Accessories	272.78	6	0.24	-73.22	37	2022-01	
1	ORD-1001	2022-01-02 11:03:43.647294	North	Furniture	Tables	280.25	7	0.29	208.67	69	2022-01	
2	ORD-1002	2022-01-03 22:07:27.294589	North	Technology	Machines	314.47	4	0.02	93.94	33	2022-01	
3	ORD-1003	2022-01-05 09:11:10.941883	North	Technology	Accessories	43.54	7	0.06	260.74	63	2022-01	
4	ORD-1004	2022-01-06 20:14:54.589178	South	Furniture	Chairs	284.57	9	0.28	-14.41	60	2022-01	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print(df.dtypes)
print("\nMissing values:\n", df.isnull().sum())
print("\nUnique Regions:", df['Region'].unique())
print("Unique Categories:", df['Category'].unique())
```

```
Order_ID      object
Order_Date    datetime64[ns]
Region        object
Category      object
Sub_Category  object
Sales         float64
Quantity      int64
Discount      float64
Profit        float64
Customer_Age  int64
Month         object
dtype: object
```

Missing values:

```
Order_ID      0
Order_Date    0
Region        0
Category      0
Sub_Category  0
Sales         0
Quantity      0
```

```
Discount      0
Profit         0
Customer_Age   0
Month          0
dtype: int64
```

```
Unique Regions: ['North' 'South' 'East' 'West']
Unique Categories: ['Technology' 'Furniture' 'Office Supplies']
```

```
total_sales = df['Sales'].sum()
total_profit = df['Profit'].sum()
total_orders = df['Order_ID'].nunique()
avg_order_value = total_sales / total_orders
total_customers_proxy = df['Customer_Age'].count() # proxy since no customer_id
```

```
print("="*40)
print("KEY PERFORMANCE INDICATORS (KPIs)")
print("="*40)
print(f"Total Sales: {total_sales}")
print(f"Total Profit: {total_profit}")
print(f"Total Orders: {total_orders}")
print(f"Avg Order Value: {avg_order_value}")
print(f"Total Transactions : {total_customers_proxy}")
```

```
=====
KEY PERFORMANCE INDICATORS (KPIs)
=====
Total      Sales   : $156,905.83
Total      Profit  : $27,040.13
Total Orders   : 500
Avg Order Value : $313.81
Total Transactions : 500
```

```
kpi_html = f"""
<div style="display:flex; gap:20px;">
  <div style="background:#4CAF50;color:white;padding:20px;border-radius:10px;flex:1;text-align:center;">
    <h3>Total Sales</h3><h2>${total_sales:,.0f}</h2>
  </div>
  <div style="background:#2196F3;color:white;padding:20px;border-radius:10px;flex:1;text-align:center;">
    <h3>Total Profit</h3><h2>${total_profit:,.0f}</h2>
  </div>
  <div style="background:#FF9800;color:white;padding:20px;border-radius:10px;flex:1;text-align:center;">
    <h3>Total Orders</h3><h2>{total_orders}</h2>
  </div>
  <div style="background:#9C27B0;color:white;padding:20px;border-radius:10px;flex:1;text-align:center;">
    <h3>Avg Order Value</h3><h2>{avg_order_value:,.0f}</h2>
  </div>
</div>
```

```
</div>
"""
display(HTML(kpi_html))
```

Total Sales
\$156,906

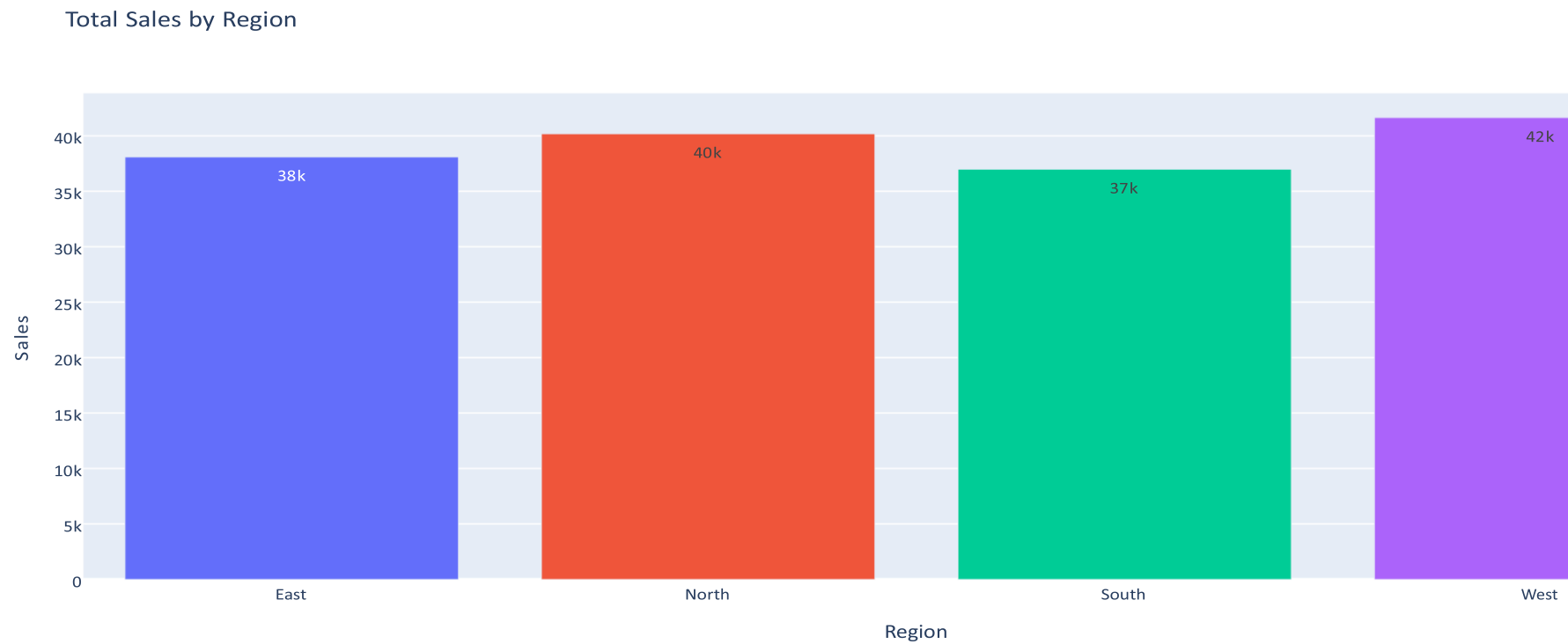
Total Profit
\$27,040

Total Orders
500

Avg Order Value
\$314

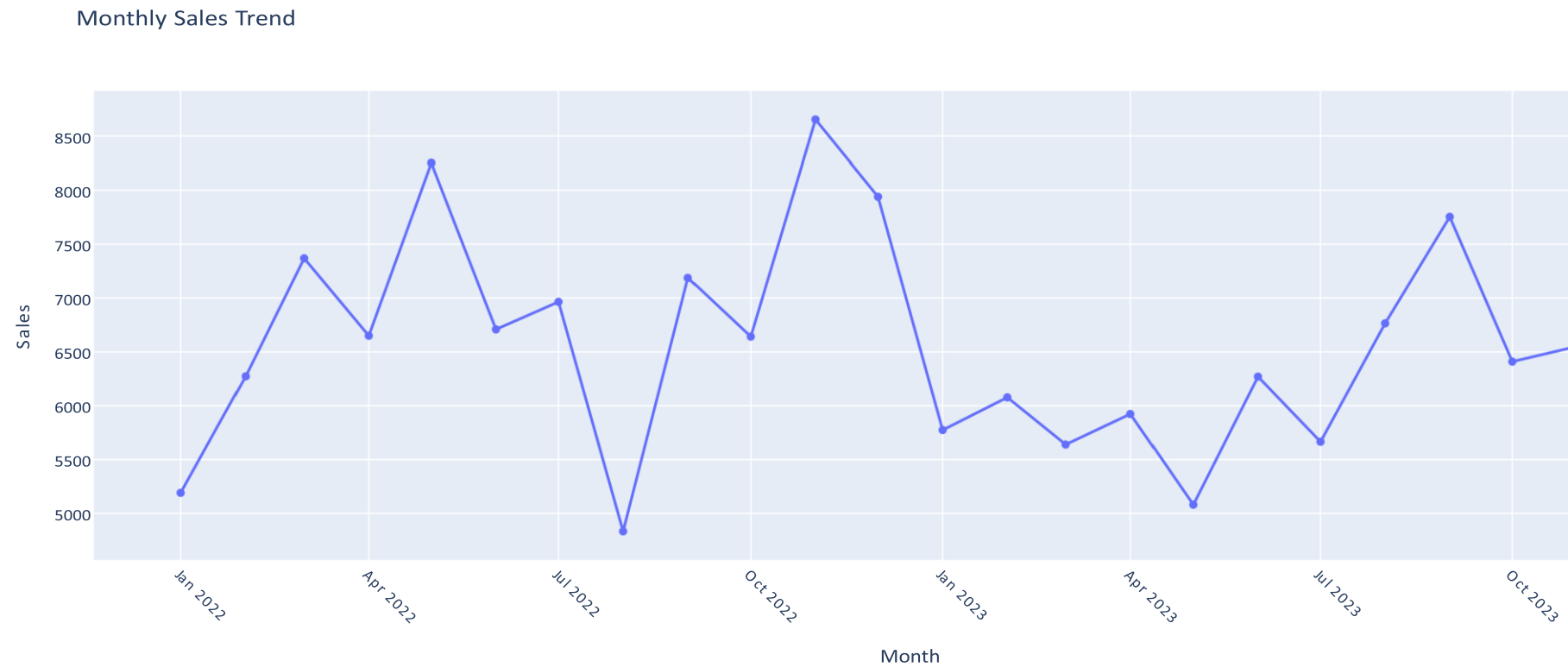
```
region_sales = df.groupby('Region', as_index=False)['Sales'].sum()

fig = px.bar(region_sales, x='Region', y='Sales', color='Region',
             title='Total Sales by Region', text_auto='.2s')
fig.update_layout(showlegend=False)
fig.show()
```



```
monthly_sales = df.groupby('Month', as_index=False)['Sales'].sum()

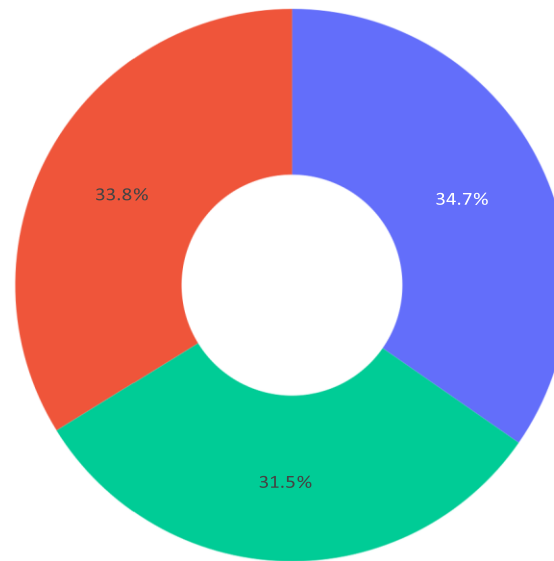
fig = px.line(monthly_sales, x='Month', y='Sales', markers=True,
              title='Monthly Sales Trend')
fig.update_xaxes(tickangle=45)
fig.show()
```



```
category_sales = df.groupby('Category', as_index=False)['Sales'].sum()

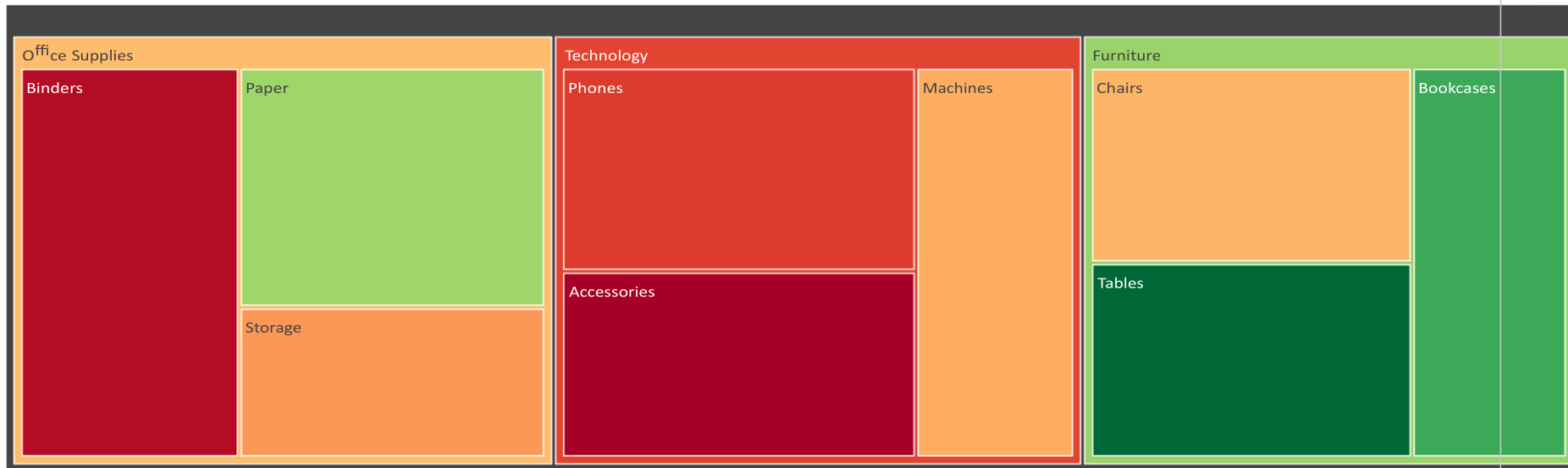
fig = px.pie(category_sales, names='Category', values='Sales',
              title='Sales Contribution by Category', hole=0.4)
fig.show()
```

Sales Contribution by Category



```
fig = px.treemap(df, path=['Category', 'Sub_Category'], values='Sales',  
                 color='Profit', color_continuous_scale='RdYlGn',  
                 title='Sales Treemap by Category & Sub-Category')  
fig.show()
```

Sales Treemap by Category & Sub-Category



```
region_dropdown = widgets.Dropdown(
    options=['All'] + list(df['Region'].unique()),
    value='All', description='Region:'
)
category_dropdown = widgets.Dropdown(
    options=['All'] + list(df['Category'].unique()),
    value='All', description='Category:'
)

def update_dashboard(region, category):
    filtered = df.copy()
    if region != 'All':
        filtered = filtered[filtered['Region'] == region]
    if category != 'All':
        filtered = filtered[filtered['Category'] == category]

    print(f"Filtered Sales: ${filtered['Sales'].sum():.2f} | ")
```

```
f"Filtered Profit: ${filtered['Profit'].sum():.2f} | "  
f"Orders: {filtered['Order_ID'].nunique()}")
```

```
fig = px.bar(filtered.groupby('Sub_Category', as_index=False)['Sales'].sum(),  
             x='Sub_Category', y='Sales', title='Sales by Sub-Category (Filtered)')  
fig.show()
```

```
interact(update_dashboard, region=region_dropdown, category=category_dropdown)
```

Region:

Category:

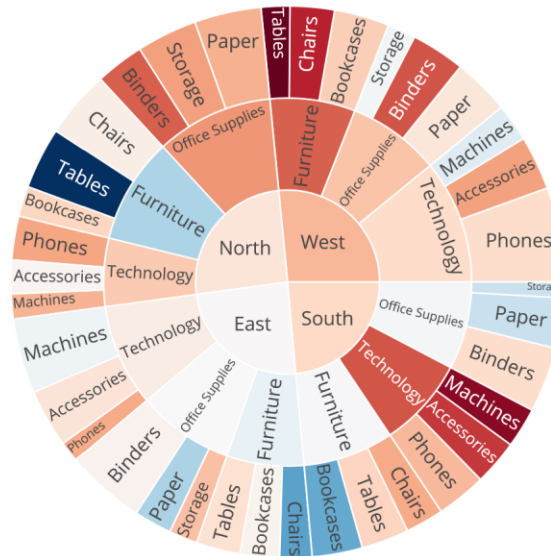
Filtered Sales: \$156,905.83 | Filtered Profit: \$27,040.13 | Orders: 500

```
update_dashboard  
def update_dashboard(region, category)
```

```
<no docstring>
```

```
fig = px.sunburst(df, path=['Region', 'Category', 'Sub_Category'], values='Sales',
                 color='Profit', color_continuous_scale='RdBu',
                 title='Drill-Down: Region → Category → Sub-Category')
fig.show()
```

Drill-Down: Region → Category → Sub-Category



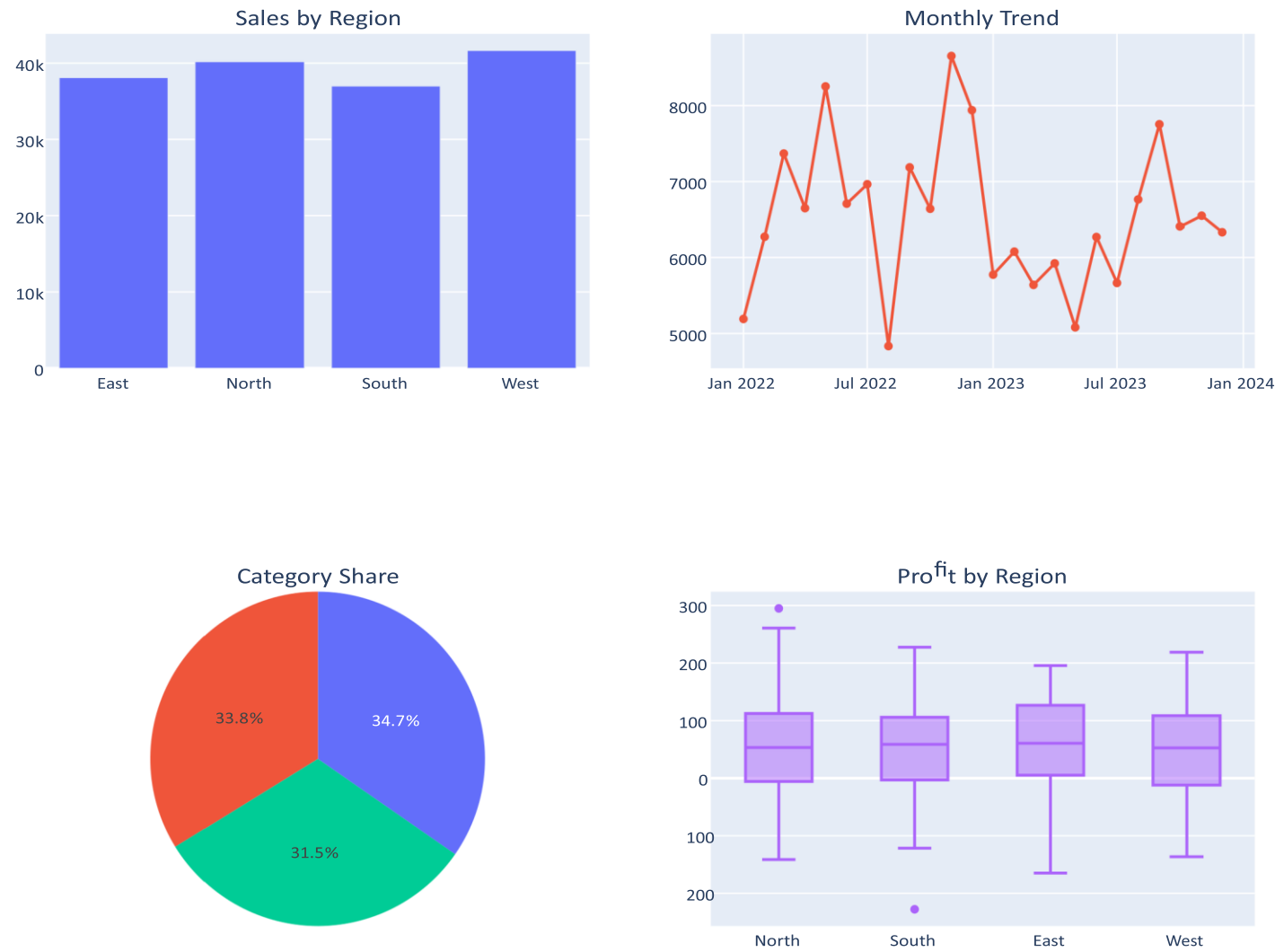
```
from plotly.subplots import make_subplots
```

```
fig = make_subplots(rows=2, cols=2,
                    subplot_titles=('Sales by Region', 'Monthly Trend', 'Category Share', 'Profit by Region'),
                    specs=[[{"type": "bar"}, {"type": "scatter"}],
                          [{"type": "pie"}, {"type": "box"}]])
```

```
fig.add_trace(go.Bar(x=region_sales['Region'], y=region_sales['Sales'], name='Sales'), row=1, col=1)
fig.add_trace(go.Scatter(x=monthly_sales['Month'], y=monthly_sales['Sales'], mode='lines+markers', name='Trend'), row=1, col=2)
fig.add_trace(go.Pie(labels=category_sales['Category'], values=category_sales['Sales'], name='Category'), row=2, col=1)
fig.add_trace(go.Box(x=df['Region'], y=df['Profit'], name='Profit'), row=2, col=2)
```

```
fig.update_layout(height=800, width=1000, title_text="Superstore Sales Performance Dashboard", showlegend=False)
fig.show()
```

Superstore Sales Performance Dashboard



```

top_region = region_sales.loc[region_sales['Sales'].idxmax(), 'Region']
best_category = category_sales.loc[category_sales['Sales'].idxmax(), 'Category']
best_subcat = df.groupby('Sub_Category')['Profit'].sum().idxmax()
worst_subcat = df.groupby('Sub_Category')['Profit'].sum().idxmin()

print("BUSINESS INSIGHTS SUMMARY")
print("="*50)
print(f"1. Top-performing region by sales: {top_region}")
print(f"2. Best-selling category: {best_category}")
print(f"3. Highest-profit sub-category: {best_subcat}")
print(f"4. Lowest-profit sub-category: {worst_subcat} (needs review)")
print(f"5. Total Sales: ${total_sales:,.2f} | Total Profit: ${total_profit:,.2f}")

```

```

BUSINESS INSIGHTS SUMMARY
=====
1. Top-performing region by sales: West
2. Best-selling category: Office Supplies
3. Highest-profit sub-category: Chairs
4. Lowest-profit sub-category: Storage (needs review)
5. Total Sales: $156,905.83 | Total Profit: $27,040.13

```

Question & Answer Section

Q1. What is Business Intelligence (BI)? How does it support organizational decision-making?

Business Intelligence refers to the technologies, processes, and tools used to collect, integrate, analyze, and present business data to support better decision-making. It converts raw operational data into actionable insights through reports and dashboards, allowing managers to identify trends, monitor performance, and make evidence-based strategic decisions rather than relying on intuition.

Q2. What is an interactive dashboard? How is it different from a static report?

An interactive dashboard is a visual interface that allows users to filter, drill down, hover for details, and explore data dynamically in real time. A static report, by contrast, presents fixed data at a single point in time with no ability to interact — the viewer can only read what's already displayed. Dashboards let users answer new questions on the fly, while static reports require a new report to be generated for each new question.

Q3. Compare Tableau and Microsoft Power BI based on features, usability, and business applications.

Tableau is known for superior visual customization, handling large/complex datasets smoothly, and being strong in advanced analytics — but it comes at a higher cost and has a steeper learning curve. Power BI is tightly integrated with the Microsoft ecosystem (Excel, Azure, Teams), is generally more affordable (especially for smaller teams), and has an easier learning curve with DAX for calculations. Tableau is often preferred by large enterprises needing rich, custom visualizations, while Power BI is favored by organizations already using Microsoft tools who want a cost-effective, easy-to-adopt BI solution.

Q4. What are Key Performance Indicators (KPIs)? Give examples of KPIs used in sales analytics.

KPIs are measurable values that indicate how effectively an organization is achieving key business objectives. In sales analytics, common KPIs include: Total Sales Revenue, Total Profit, Number of Orders, Average Order Value, Customer Acquisition Count, Sales Growth Rate, and Profit Margin.

Q5. Explain the purpose of filters, slicers, and drill-down functionality in dashboards.

Filters and slicers let users narrow down the displayed data to a specific subset (e.g., only "West" region or a particular date range) without altering the underlying dataset, making analysis more focused and relevant. Drill-down functionality allows users to move from a summary level (e.g., yearly sales) to more granular detail (e.g., monthly, then daily sales) within the same visual, enabling deeper investigation of specific trends or anomalies.

Q6. What factors should be considered while designing an effective business dashboard?

Key factors include: identifying the target audience and their specific needs, choosing the right KPIs and visuals for the objective, maintaining a clean and uncluttered layout, ensuring consistent color schemes and formatting, prioritizing the most important metrics at the top, using appropriate chart types for the data, and testing interactivity (filters, drill-downs) for usability.

Q7. How can dashboards help managers monitor organizational performance in real time?

Dashboards connected to live or frequently refreshed data sources allow managers to view up-to-date KPIs and metrics without waiting for manual reports. This enables faster identification of issues (e.g., a sudden sales drop) and quicker corrective action, supporting agile, real-time decision-making rather than reacting to outdated monthly reports.

Q8. Why is data visualization important in Business Intelligence tools?

Visualization is at the core of BI because it converts large volumes of raw data into intuitive graphical formats that are easy to interpret quickly. It helps non-technical stakeholders understand complex data without needing to analyze spreadsheets, and it highlights patterns, outliers, and trends that support faster and more accurate business decisions.

Q9. What insights can be derived from a sales dashboard for an e-commerce company?

A sales dashboard can reveal: best-selling products/categories, highest-revenue regions or customer segments, seasonal or monthly sales trends, the impact of discounts on profit margins, average order value trends, and underperforming products or regions that may need marketing attention or discontinuation.

Q10. Explain how interactive dashboards improve communication between data analysts and business stakeholders.

Interactive dashboards allow business stakeholders to explore data themselves — filtering by region, time, or category — without needing an analyst to generate a new report each time. This self-service capability reduces back-and-forth communication delays, empowers non-technical users to answer their own questions, and lets analysts focus on deeper analysis rather than repetitive reporting requests, ultimately speeding up decision-making across the organization.